Algorithms & Data Structures Project - KD-Trees

Alastair Watt

Semester 1 2025/26

Contents

1 Introduction **2**

2 Dataset **2**
2.1 Preparation of the Set 2
2.2 Implementation 3

3 Kd-trees **10**
3.1 Trees and Binary Search Trees 10
3.2 Extending to 2d-trees then Kd-trees 11

4 Implementing Kd-tree **12**
4.1 Code 12

5 Performance Evaluation **23**
5.1 Outline of naive searches 23
5.2 Implementation and comparison 23
5.3 Curse of Dimensionality 37
5.4 Other analysis 37

6 Applications **38**
6.1 How could this be used on a trading floor? 38
6.2 Next steps 38

7 Conclusion **39**

A Definitions of Financial Terms **40**

1 Introduction

As my overall degree is a Joint Major of Economics and Mathematics I have chosen to bring economics into the project by incorporating real-life financial data for my data set. It is however a Data Structures and C project, hence I have tried to limit this finance aspect to just the data set and a brief applications section. I will begin by collating, preparing and normalising my data set for use within the Kd-tree. Before implementing the tree itself I will give my best interpretation of a Kd-tree and the concept as a whole. After the tree is operational I will use all 12 metrics and this will allow me to compare performance to each other and to naïve search while also exploring the curse of dimensionality.

2 Dataset

To start the project off I had to collate my dataset from real-life equity data. I collated and chose 2000 of the largest firms on the American stock exchange (excluding banks as they do not publish all the required data) and collected figures on a range of different data points, some of which were MarketCap, P/E, EBITDA Margin (%) along with many others. I chose these due to their specific importance to valuing firms and because I think they give the broadest measures for the least amount of data. This is a coding and data structures project and as such I tried to find the balance of focusing on the content while also having a proper workable dataset. I have decided on 12 metrics and these metrics will be our 'dimensions' later on when we construct the Kd-tree.

All data was sourced from stock analysis (1) https://stockanalysis.com from the 12th of December. *Year on year metrics have varying dates of collection i.e. YoY from Sept 30th or YoY Oct 30th etc*

2.1 Preparation of the Set

I then decided that I would use the classic standardisation for my normalisation method. My reasoning for this is as follows; some data points consisted of very large positive integers that were heavily skewed, in general, natural scales were widely different across variables and as Kd-trees rely on Euclidean distance this means that each features numeric variation is on a comparable scale. Using these standard formulas;

For one feature/column X with values $x_1, x_2, \ldots, x_n$:

1. **Mean (average)**

$$\mu = \frac{1}{n}\sum_{i=1}^{n} x_i$$

2. **Standard deviation (population)**

$$\sigma = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(x_i - \mu)^2}$$

3. **Standardised value (z-score) for each data point x_i**

$$z_i = \frac{x_i - \mu}{\sigma}$$

I use the population standard deviation because I standardise the entire dataset of 2000 firms for similarity search (rather than estimating population parameters from a smaller sample).

2.2 Implementation

I have included my code below: CSV_Parsing_and_Standardisation.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h> // string utilities
#include <stdbool.h> // bool types
#include <math.h> // math functions
#include <ctype.h> // character classification functions

// Helper: Remove UTF-8 BOM if present at start of line
static void removeBOM(char *line) {
    unsigned char *u = (unsigned char*) line;
    // Check for BOM byte sequence 0xEF, 0xBB, 0xBF
    if (u[0] == 0xEF && u[1] == 0xBB && u[2] == 0xBF) {
        // Shift the string 3 bytes to the left to remove BOM
        line[0] = line[3];
        line[1] = line[4];
        line[2] = line[5];
        // Use memmove to shift the remaining characters (including null terminator) over the BOM
        memmove(line, line + 3, strlen(line + 3) + 1);
    }
}

// Helper: Trim newline and carriage return characters from end of line (in-place)
static void chomp(char *line) {
    size_t len = strlen(line);
    if (len > 0 && (line[len - 1] == '\n' || line[len - 1] == '\r')) {
        line[--len] = '\0'; // remove trailing newline or CR
    }
    if (len > 0 && line[len - 1] == '\r') { // handle CR if CRLF
        line[--len] = '\0';
    }
}

// Helper: Determine if a field looks like a stock ticker (all uppercase letters/digits and short length)
static bool looks_like_ticker(const char *s) {
    if (!s) return false;
    // Skip leading whitespace if any
    while (*s && *s == ' ') s++;
    int len = 0;
    for (const char *p = s; *p; p++) {
        if (*p == ' ' || *p == '\t') {
            // If spaces in the string, it's likely a company name, not a ticker
            return false;
        }
        if (!(*p >= '0' && *p <= '9') && !(*p >= 'A' && *p <= 'Z')) {
            // Contains a character that is not a digit or uppercase letter
            return false;
        }
        len++;
    }
    // Typical ticker symbols are short (e.g., 1-5 characters)
    return (len > 0 && len <= 5);
}

// Helper: Split a CSV line into fields, handling quoted commas. Modifies the line buffer in-place.
static int splitCSVLine(char *line, char **fields, int maxFields) {
    bool inQuote = false;
    int fieldCount = 0;
```

```
    char *p = line;
    char *fieldStart = line;
    while (*p && fieldCount < maxFields) {
        if (*p == '\"') {
            // Toggle quote state or handle escaped quote
            if (inQuote && *(p+1) == '\"') {
                // Double quote inside a quoted field -> treat as literal quote
                // Remove one quote by shifting the rest of the string left
                memmove(p, p+1, strlen(p));
                // After removal, keep inQuote true and advance pointer
                p++;
                continue;
            } else {
                inQuote = !inQuote; // entering or leaving quoted section
                // Remove the quote character by shifting left
                memmove(p, p+1, strlen(p));
                continue; // continue without advancing p (since we shifted contents)
            }
        }
        if (!inQuote && *p == ',') {
            // End of field reached (comma outside quotes)
            *p = '\0'; // terminate the current field
            if (fieldCount < maxFields) {
                fields[fieldCount++] = fieldStart; // save field start pointer
            }
            fieldStart = p + 1; // next field begins after comma
        }
        p++;
    }
    // Add the last field
    if (*p == '\0' && fieldStart && fieldCount < maxFields) {
        fields[fieldCount++] = fieldStart;
    }
    return fieldCount;
}

// Helper: Parse a string into a double, allowing commas, percent signs, and parentheses for negatives.
// Returns true if a number was parsed successfully, false if the string is empty or non-numeric.
static bool parseDoubleLoose(char *field, double *outVal) {
    if (!field) return false;
    // Trim leading and trailing whitespace
    while (*field && isspace((unsigned char)*field)) field++;
    if (*field == '\0') {
        return false; // empty or all whitespace
    }
    // Handle negative indicated by parentheses e.g. "(123)"
    bool negative = false;
    char *end = field + strlen(field) - 1;
    if (*field == '(' && *end == ')') {
        negative = true;
        field++;       // skip '('
        *end = '\0';   // remove ')'
        // Trim any whitespace after removing parenthesis
        while (*end == '\0' && end > field && isspace((unsigned char)*(end-1))) {
            *(--end) = '\0';
        }
    }
    // Remove percentage sign if present at end
    size_t len = strlen(field);
    if (len > 0 && field[len-1] == '%') {
        field[len-1] = '\0';
    }
    // Remove any commas (thousands separators) and internal whitespace from the field
```

```c
    char *src = field;
    char *dst = field;
    while (*src) {
        if (*src != ',' && !isspace((unsigned char)*src)) {
            *dst++ = *src;
        } else if (isspace((unsigned char)*src)) {
            // Skip internal spaces (often grouping separators in numbers)
        }
        src++;
    }
    *dst = '\0'; // null-terminate after removing characters
    if (*field == '\0') {
        return false; // nothing left to parse after cleanup
    }
    // Use strtod to parse the cleaned string
    char *endptr;
    double val = strtod(field, &endptr);
    if (endptr == field) {
        return false; // no numeric conversion performed
    }
    // If there are leftover characters that are not just whitespace, treat parse as incomplete
    while (*endptr && isspace((unsigned char)*endptr)) endptr++;
    if (*endptr != '\0') {
        // Leftover non-space characters indicate a malformed number
        return false;
    }
    if (negative) {
        val = -val;
    }
    if (outVal) {
        *outVal = val;
    }
    return true;
}

// Opens CSV, counts data rows and total columns. Skips header row if present.
int countCSV(const char *filename, int *rows, int *cols) {
    FILE *fp = fopen(filename, "r");
    if (!fp) return -1; // error opening file
    char line[2048];
    *rows = 0;
    *cols = 0;
    bool firstLine = true;
    bool headerDetected = false;
    // Read through the file line by line
    while (fgets(line, sizeof(line), fp)) {
        if (firstLine) {
            removeBOM(line); // remove BOM on the first line if present
            firstLine = false;
        }
        // Skip empty lines (only newline or carriage return characters)
        if (strspn(line, "\r\n") == strlen(line)) {
            continue;
        }
        if (*cols == 0) {
            // This is the first non-empty line: determine column count
            int count = 1;
            for (char *p = line; *p; p++) {
                if (*p == ',') count++; // count commas to estimate number of fields
            }
            *cols = count;
            // Determine if this first line is a header by checking if third field is non-numeric
            char *fields[256];
```

```
            int nf = splitCSVLine(line, fields, 256);
            if (nf >= 3) {
                double tmp;
                if (!parseDoubleLoose(fields[2], &tmp)) {
                    headerDetected = true;
                }
            }
            if (!headerDetected) {
                // If it's not a header, count it as a data row
                (*rows)++;
            }
        } else {
            // Subsequent non-empty lines (header already handled if any)
            (*rows)++;
        }
    }
    fclose(fp);
    return 0;
}

// Parse the CSV file into arrays of company names, tickers, numeric data and missing masks.
// Assumes company_names, tickers, data, missing are pre-allocated to the given sizes (rows x numeric_cols).
int parseCSV(const char *filename, char **company_names, char **tickers, double **data, bool **missing, int max_rows, int numeric_cols) {
    FILE *fp = fopen(filename, "r");
    if (!fp) return -1;
    char line[2048];
    int r = 0;
    bool header_skipped = false;
    bool tickerFirst = false;
    // Read lines until EOF or until we've read max_rows data lines
    while (r < max_rows && fgets(line, sizeof(line), fp)) {
        if (!header_skipped) {
            removeBOM(line); // remove BOM on first line if present
        }
        chomp(line); // remove newline and CR at end
        if (line[0] == '\0') {
            continue; // skip blank line
        }
        if (!header_skipped) {
            // On the first non-empty line, decide if it's header or data
            char *fields[256];
            int fieldCount = splitCSVLine(line, fields, 256);
            if (fieldCount >= 3) {
                double tmp;
                if (!parseDoubleLoose(fields[2], &tmp)) {
                    // This line is a header (third column not numeric)
                    // Determine order of columns from header labels
                    if (strcasecmp(fields[0], "Symbol") == 0 || strcasecmp(fields[0], "Ticker") == 0) {
                        tickerFirst = true;
                    } else if (strcasecmp(fields[1], "Company Name") == 0 || strcasecmp(fields[1], "Name") == 0) {
                        tickerFirst = true;
                    } else {
                        tickerFirst = false;
                    }
                    header_skipped = true;
                    // Skip header line and continue without incrementing r
                    continue;
                }
            }
            // If it was not identified as header, use a heuristic to decide column order for data
```

```
            if (fieldCount >= 2) {
                tickerFirst = looks_like_ticker(fields[0]) && !looks_like_ticker(fields[1]);
            } else {
                tickerFirst = false;
            }
            header_skipped = true;
            // Process this first data line using the fields we split
            // (No continue here, we fall through to data processing below)
        }
        // Now handle the current line as a data row
        char *fields[256];
        int fieldCount = splitCSVLine(line, fields, 256);
        if (fieldCount < 2) {
            continue; // skip malformed line with too few fields
        }
        // Assign company name and ticker based on identified order
        const char *firstField = fields[0];
        const char *secondField = (fieldCount > 1 ? fields[1] : "");
        if (tickerFirst) {
            // First field is ticker, second field is company name
            tickers[r] = strdup(firstField);
            company_names[r] = strdup(secondField);
        } else {
            // First field is company name, second is ticker
            company_names[r] = strdup(firstField);
            tickers[r] = strdup(secondField);
        }
        // Parse numeric fields from the third column onward
        for (int c = 0; c < numeric_cols; ++c) {
            int fieldIndex = 2 + c;
            double value;
            if (fieldIndex < fieldCount && parseDoubleLoose(fields[fieldIndex], &value)) {
                data[r][c] = value;
                missing[r][c] = false; // valid number parsed
            } else {
                // Missing or invalid numeric field
                data[r][c] = 0.0;
                missing[r][c] = true;
            }
        }
        // If fewer numeric fields were present than expected, mark the rest as missing
        if (fieldCount - 2 < numeric_cols) {
            for (int c = fieldCount - 2; c < numeric_cols; ++c) {
                data[r][c] = 0.0;
                missing[r][c] = true;
            }
        }
        r++; // increment data row count
    }
    fclose(fp);
    return r;
}

// Compute column means ignoring missing values. Returns an array of means for each column.
double* computeColumnMeans(double **data, bool **missing, int rows, int cols) {
    double *means = malloc(cols * sizeof(double)); // allocate means array
    for (int j = 0; j < cols; j++) { // for each column
        double sum = 0.0;
        int count = 0;
        for (int i = 0; i < rows; i++) {
            if (!missing[i][j]) {
                sum += data[i][j];
                count++;
            }
        }
```

```c
        means[j] = (count > 0) ? (sum / count) : 0.0; // compute mean or 0.0 if no valid values
    }
    return means;
}

// Compute column standard deviations (population stddev). Expects means array provided.
double* computeColumnStd(double **data, int rows, int cols, double *means) {
    double *stddev = malloc(cols * sizeof(double)); // allocate stddev array
    for (int j = 0; j < cols; j++) {
        double sumsq = 0.0;
        for (int i = 0; i < rows; i++) {
            double diff = data[i][j] - means[j];
            sumsq += diff * diff;
        }
        stddev[j] = sqrt(sumsq / (rows > 0 ? rows : 1)); // population standard deviation
    }
    return stddev;
}

// Compute z-scores for each data value. Allocates and returns a 2D array of z-scores.
double** computeZScores(double **data, int rows, int cols, double *means, double *stddev) {
    double **z = malloc(rows * sizeof(double*));
    for (int i = 0; i < rows; i++) {
        z[i] = malloc(cols * sizeof(double));
        for (int j = 0; j < cols; j++) {
            if (stddev[j] > 0.0) {
                z[i][j] = (data[i][j] - means[j]) / stddev[j];
            } else {
                z[i][j] = 0.0;
            }
        }
    }
    return z;
}

// Print the standardized data (names and z-scores) in CSV form
void printStandardized(char **company_names, char **tickers, double **zscores, int rows, int cols) {
    for (int i = 0; i < rows; i++) {
        printf("%s (%s)", company_names[i], tickers[i]); // print company name and ticker symbol
        for (int j = 0; j < cols; j++) {
            printf(",%f", zscores[i][j]); // print z-score for each metric
        }
        printf("\n");
    }
}

// Main function to drive the standardization process (if built as standalone program)
#ifndef BUILD_AS_LIBRARY
int main(int argc, char *argv[]) {
    if (argc < 2) {
        fprintf(stderr, "Usage: %s data.csv [-p]\n", argv[0]);
        return 1;
    }
    const char *filename = argv[1];
    bool doPrint = false;
    if (argc >= 3 && (strcmp(argv[2], "-p") == 0 || strcmp(argv[2], "--print") == 0)) {
        doPrint = true;
    }
    int totalRows = 0, totalCols = 0;
    if (countCSV(filename, &totalRows, &totalCols) != 0 || totalCols < 2) {
        fprintf(stderr, "Error: could not read file or invalid format.\n");
        return 1;
```

```
    }
    int numeric_cols = totalCols - 2; // number of numeric columns
    // Allocate arrays for data
    char **company_names = malloc(totalRows * sizeof(char*));
    char **tickers = malloc(totalRows * sizeof(char*));
    double **data = malloc(totalRows * sizeof(double*));
    bool **missing = malloc(totalRows * sizeof(bool*));
    for (int i = 0; i < totalRows; i++) {
        data[i] = malloc(numeric_cols * sizeof(double));
        missing[i] = malloc(numeric_cols * sizeof(bool));
    }
    // Parse the CSV file into the arrays
    int rowsRead = parseCSV(filename, company_names, tickers, data, missing, totalRows,
        numeric_cols);
    if (rowsRead <= 0) {
        fprintf(stderr, "No data parsed.\n");
        return 1;
    }
    // Compute means and replace missing values with mean for each column
    double *means = computeColumnMeans(data, missing, rowsRead, numeric_cols);
    for (int i = 0; i < rowsRead; i++) {
        for (int j = 0; j < numeric_cols; j++) {
            if (missing[i][j]) {
                data[i][j] = means[j];
            }
        }
    }
    // Compute standard deviations and z-scores
    double *stddev = computeColumnStd(data, rowsRead, numeric_cols, means);
    double **zdata = computeZScores(data, rowsRead, numeric_cols, means, stddev);
    if (doPrint) {
        // Optionally print the standardized data (with company name and ticker)
        printStandardized(company_names, tickers, zdata, rowsRead, numeric_cols);
    }
    // Free allocated memory
    for (int i = 0; i < totalRows; i++) {
        free(company_names[i]);
        free(tickers[i]);
        free(data[i]);
        free(missing[i]);
        if (i < rowsRead) free(zdata[i]);
    }
    free(company_names);
    free(tickers);
    free(data);
    free(missing);
    free(zdata);
    free(means);
    free(stddev);
    return 0;
}
#endif
```

In summary this code does the following;

- Opens, counts and parses the rows and columns of a CSV file.
- Notes missing values and fills them with the mean value.
- Computes column standard deviations.
- Computes z-scores and stores them in arrays.

Compile instructions: only to print z-score values.

- gcc CSV_Parsing_and_Standardisation.c -o Standardise.o
- ./Standardise.o Data_for_Project.csv -p

3 Kd-trees

3.1 Trees and Binary Search Trees

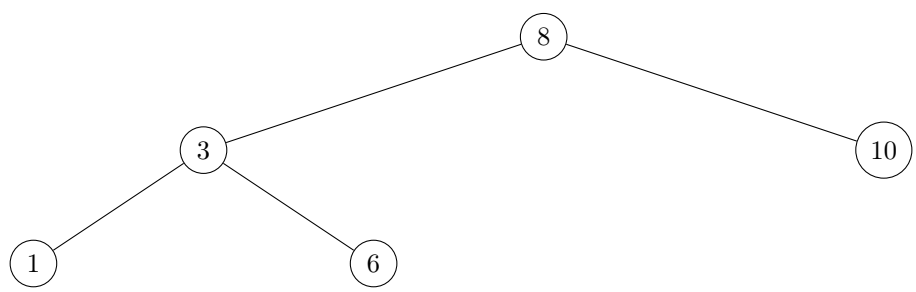
A tree is a hierarchical data structure (very similar to an actual family tree) that organises data in a parent-child relationship. Intuitively the parents comes first and the children branch out from them. Each individual in a would-be family tree (or data point in this data structure) is called a node with each node having child nodes below it. Naturally, child nodes can become parent nodes in turn if they have child nodes below themselves and so on. Nodes that do not have any children are called leaves and the first or topmost node is referred to as the root.

This structure forms a hierarchy of data rather than just a linear sequence of elements.

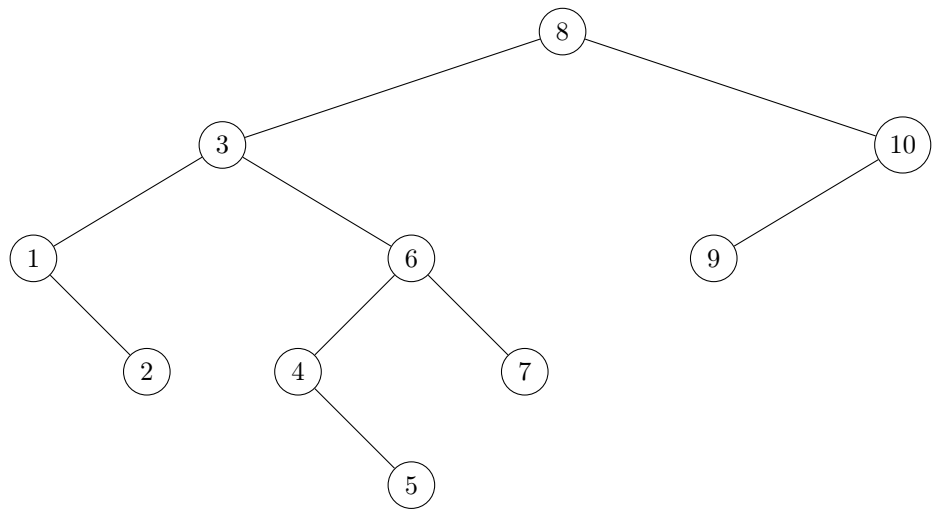
| Binary tree: a special kind of tree where each node has zero, one or two (at most 2) children. These are referred to as the left and right child. This subset of trees offers a useful maximum branching factor of two at each node. |

A binary search tree (BST) is a binary tree, a concept I have just looked at above, with a property that allows for efficient searching. Each node in a BST stores a value and is called a key. The structure is arranged such that all keys in the left subtree are less than their parent's key and all the keys in the right subtree are greater. This property holds recursively for each subtree. Consider inserting a few numbers into a BST.

- Insert 8, it becomes the root.
- Insert 3, $3 < 8$ so it goes to the left of 8.
- Insert 10, $10 > 8$ so it goes to the right of 8.
- Insert 1, $1 < 8$ so it goes left of 8. Also $1<3$ so it becomes the left child of 3.
- Insert 6, $6 < 8$ so it goes to the left. But $6>3$ so it becomes the right child of 3.



Then inserting 9, 4, 5, 7 and 2 we have:



The above BST's are structured in a simple manner. At each node, left values are smaller and right values are larger. This enables a binary search on the tree (hence the name). To find a value, you compare it to the root and then recur left or right , halving the search space at each step. In a completely balanced BST, searching or any other operation takes on average O(log n) time since each comparison enables the operation to skip half of the remaining tree. In the worst case however, the performance degrades to O(n). Imagine a tree that begins to the left and continues in a straight line i.e. every value is less than the one before, this is essentially a linear scan.

3.2 Extending to 2d-trees then Kd-trees

In a 2D Space, each data point has two coordinates, like an x and y value. A 2D kd-tree is a binary tree that alternates between splitting the data. We handle points on a plane like (x,y) pairs.

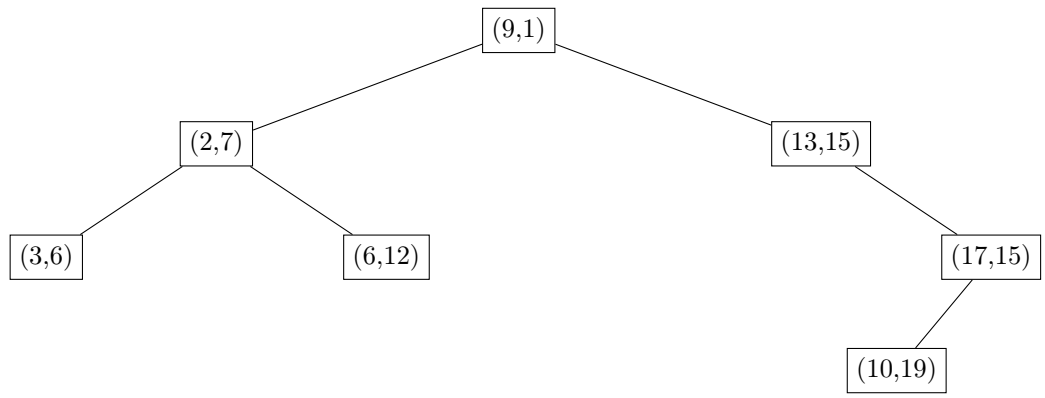
With the following points a 2d-tree is built like this; (3,6), (17,15), (13,15), (6,12), (9,1), (2,7), (10,19).

Level 0: We find the median value of the x-coordinates, which is 9 in this case and as such the root will be (9,1). All points with x $<$ 9 go left and all points with x $>=$ 9 go right.

Level 1: Left subtree consists of the points; (3,6), (6,12), (2,7). Sort by y and we get 7 as the median hence (2,7) becomes the left child of the root. $6 < 7$ so (3,6) goes to the left and as $12 > 7$, (6,12) goes to the right. – Right subtree consists of the points: (17,15), (13,15), (10,19). Sort by y and we get the median as 15 and we pick (13,15) to become the right child of the root. $15 = 15$ so (17,15) goes to the right and $19 > 15$ so (10,19) goes to the right.

Level 2: We switch back to x and are only concerned with the right subtree. As $17 > 10$ (17,15) becomes the right child of (13,15) and (10,19) goes to the left.

This 2d-tree is created below.



This process can intuitively be generalised to K-dimensions. Let's say we had 4 dimensions x, y, z and h. First you would cycle through x by finding the median and pushing all lesser values to the left and greater values to the right. Then you would do the same for y, z and then h before circling back to x if they are more values and repeating the process.

4 Implementing Kd-tree

4.1 Code

My code is below: Build_KD-Tree_and_NNSearch.c

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>
#include <ctype.h>
#include <math.h>

/*
  This file is a separate program that uses the data parsing and standardization functions
  from CSV_Parsing_Standardisation.c. It leverages these functions:
    - countCSV
    - parseCSV
    - computeColumnMeans
    - computeColumnStd
    - computeZScores

  This program focuses on:
    - interactive prompts
    - metric subset selection (1..12 or 'all')
    - query by ticker OR company name
    - building a k-d tree in k dimensions (k = number of selected metrics)
    - Euclidean nearest-neighbour search

*/

#define MAX_METRICS 12
#define LINE_BUF    16384

// Calling functions from Data_Project_FinalWTicker.C (external parsing and standardization)
```

```c
int     countCSV(const char *filename, int *rows, int *cols);
int     parseCSV(const char *filename, char **company_names, char **tickers,
                 double **data, bool **missing, int rows, int numeric_cols);
double* computeColumnMeans(double **data, bool **missing, int rows, int cols);
double* computeColumnStd(double **data, int rows, int cols, double *means);
double** computeZScores(double **data, int rows, int cols, double *means, double *stddev);

// Small utilities for safe memory allocation
static void *xmalloc(size_t n) { // allocate memory or exit on failure
    void *p = malloc(n);
    if (!p) {
        fprintf(stderr, "Out of memory.\n");
        exit(2);
    }
    return p;
}

static void *xcalloc(size_t n, size_t s) { // allocate zero-initialized memory or exit on failure
    void *p = calloc(n, s);
    if (!p) {
        fprintf(stderr, "Out of memory.\n");
        exit(2);
    }
    return p;
}

static char *xstrdup(const char *s) { // duplicate string, exit on failure if null
    if (!s) return NULL;
    char *p = malloc(strlen(s) + 1);
    if (!p) {
        fprintf(stderr, "Out of memory.\n");
        exit(2);
    }
    strcpy(p, s);
    return p;
}

// Definition of the Stock struct (from earlier context)
typedef struct {
    char *name;
    char *ticker;
    double z[MAX_METRICS]; // standardized metrics
} Stock;

// Dataset structure to hold all stocks and related info
typedef struct {
    Stock *stocks;
    int n_rows;
    int n_metrics;           // number of metrics (<= MAX_METRICS)
    char *metric_names[MAX_METRICS];
} Dataset;

// Free all allocated memory in a Dataset
static void dataset_free(Dataset *ds) {
    if (!ds) return;
    if (ds->stocks) {
        for (int i = 0; i < ds->n_rows; i++) {
            free(ds->stocks[i].name);
            free(ds->stocks[i].ticker);
        }
        free(ds->stocks);
    }
    for (int j = 0; j < MAX_METRICS; j++) {
        free(ds->metric_names[j]);
```

```c
        ds->metric_names[j] = NULL;
    }
    memset(ds, 0, sizeof(*ds));
}

// Case-insensitive string comparison (ASCII). Returns 0 if equal ignoring case.
static int strcasecmp_ascii(const char *a, const char *b) {
    unsigned char ca, cb;
    do {
        ca = (unsigned char) *a++;
        cb = (unsigned char) *b++;
        if (ca >= 'A' && ca <= 'Z') ca = (unsigned char)(ca - 'A' + 'a');
        if (cb >= 'A' && cb <= 'Z') cb = (unsigned char)(cb - 'A' + 'a');
    } while (ca == cb && ca != '\0');
    return (ca - cb);
}

// Remove newline (and carriage return if present) from end of a string (in-place)
static void chomp(char *s) {
    if (!s) return;
    size_t len = strlen(s);
    if (len > 0 && (s[len-1] == '\n' || s[len-1] == '\r')) {
        s[--len] = '\0';
    }
    if (len > 0 && s[len-1] == '\r') {
        s[--len] = '\0';
    }
}

// Load CSV data, parse it with external parser, and standardize it into a Dataset
static Dataset load_dataset_and_standardise_via_other_file(const char *filename) {
    Dataset ds;
    memset(&ds, 0, sizeof(ds));
    int totalRows = 0, totalCols = 0;
    // Count rows and columns (header row is auto-detected in countCSV)
    if (countCSV(filename, &totalRows, &totalCols) != 0) {
        fprintf(stderr, "Error: cannot open '%s'\n", filename);
        exit(1);
    }
    if (totalCols < 3) {
        fprintf(stderr, "Error: CSV must have at least 3 columns (name/ticker + at least 1
            metric).\n");
        exit(1);
    }
    // Determine number of numeric metrics
    int numeric_cols = totalCols - 2;
    if (numeric_cols < 1) {
        fprintf(stderr, "Error: no numeric metrics found in CSV.\n");
        exit(1);
    }
    // Limit metrics to MAX_METRICS for safety
    if (numeric_cols > MAX_METRICS) numeric_cols = MAX_METRICS;
    ds.n_metrics = numeric_cols;
    // Open file to read first line for metric names
    FILE *fp = fopen(filename, "r");
    if (!fp) {
        fprintf(stderr, "Error: cannot open '%s'\n", filename);
        exit(1);
    }
    char line[LINE_BUF];
    bool has_header = false;
    // Read first non-empty line (could be header or first data)
    while (fgets(line, sizeof(line), fp)) {
        chomp(line);
        // Remove BOM if present at beginning
```

```c
        if ((unsigned char)line[0] == 0xEF && (unsigned char)line[1] == 0xBB && (unsigned char)line[2] == 0xBF) {
            memmove(line, line + 3, strlen(line + 3) + 1);
        }
        if (line[0] == '\0') continue;
        // Split the line by commas (note: assumes no quoted commas in header fields)
        char *tmpLine = xstrdup(line); // duplicate line to tokenize
        char *saveptr = NULL;
        char *token = strtok_r(tmpLine, ",", &saveptr);
        char *fields[256];
        int nf = 0;
        while (token && nf < 256) {
            // Trim whitespace around token
            while (*token && isspace((unsigned char)*token)) token++;
            char *end = token + strlen(token);
            while (end > token && isspace((unsigned char)*(end-1))) *(--end) = '\0';
            fields[nf++] = token;
            token = strtok_r(NULL, ",", &saveptr);
        }
        if (nf >= 3) {
            // Try to interpret third field as a number to detect header vs data
            char *endptr;
            double val = strtod(fields[2], &endptr);
            (void)val;
            bool numeric = (endptr != fields[2]);
            if (numeric) {
                // If parsed number, ensure trailing chars are just whitespace or '%'
                while (*endptr && isspace((unsigned char)*endptr)) endptr++;
                if (*endptr == '%') endptr++;
                while (*endptr && isspace((unsigned char)*endptr)) endptr++;
                if (*endptr != '\0') {
                    numeric = false;
                }
            }
            if (!numeric) {
                has_header = true;
            }
        }
        // Store metric names
        if (has_header) {
            // Use header labels for metric names
            for (int j = 0; j < ds.n_metrics && 2 + j < nf; j++) {
                ds.metric_names[j] = xstrdup(fields[2 + j]);
            }
        } else {
            // No header: create generic metric names
            for (int j = 0; j < ds.n_metrics; j++) {
                char buf[64];
                snprintf(buf, sizeof(buf), "Metric %d", j + 1);
                ds.metric_names[j] = xstrdup(buf);
            }
        }
        free(tmpLine);
        break;
    }
    fclose(fp);
    // totalRows (from countCSV) is the number of data rows (header excluded)
    if (totalRows <= 0) {
        fprintf(stderr, "Error: no data rows found.\n");
        exit(1);
    }
    ds.n_rows = totalRows;
    // Allocate arrays for parseCSV results
    char **company_names = (char **) xmalloc((size_t) totalRows * sizeof(char*));
    char **tickers = (char **) xmalloc((size_t) totalRows * sizeof(char*));
```

```c
    double **data = (double **) xmalloc((size_t) totalRows * sizeof(double*));
    bool **missing = (bool **) xmalloc((size_t) totalRows * sizeof(bool*));
    for (int i = 0; i < totalRows; i++) {
        data[i] = (double *) xmalloc((size_t) numeric_cols * sizeof(double));
        missing[i] = (bool *) xmalloc((size_t) numeric_cols * sizeof(bool));
    }
    // Parse CSV file into name, ticker, data, missing arrays
    int rows_read = parseCSV(filename, company_names, tickers, data, missing, totalRows,
        numeric_cols);
    if (rows_read < 0) {
        fprintf(stderr, "Error: failed to parse CSV file '%s'.\n", filename);
        exit(1);
    }
    if (rows_read == 0) {
        fprintf(stderr, "Error: no data parsed from '%s'.\n", filename);
        exit(1);
    }
    if (rows_read != totalRows) {
        // Adjust n_rows if some lines were skipped or file had fewer lines
        ds.n_rows = rows_read;
        if (ds.n_rows <= 0) {
            fprintf(stderr, "Error: no valid data rows parsed.\n");
            exit(1);
        }
        // Free any unused row buffers that were allocated beyond actual rows_read
        for (int i = ds.n_rows; i < totalRows; i++) {
            free(data[i]);
            free(missing[i]);
        }
    }
    // Allocate Stock array and transfer parsed name/ticker strings
    ds.stocks = (Stock *) xcalloc((size_t) ds.n_rows, sizeof(Stock));
    for (int i = 0; i < ds.n_rows; i++) {
        ds.stocks[i].name = company_names[i];
        ds.stocks[i].ticker = tickers[i];
    }
    // Free pointer arrays (the strings are now owned by ds.stocks)
    free(company_names);
    free(tickers);
    // Compute means and fill in missing values with the mean for each metric
    double *means = computeColumnMeans(data, missing, ds.n_rows, ds.n_metrics);
    for (int i = 0; i < ds.n_rows; i++) {
        for (int j = 0; j < ds.n_metrics; j++) {
            if (missing[i][j]) {
                data[i][j] = means[j];
            }
        }
    }
    // Compute standard deviations for each metric
    double *stddev = computeColumnStd(data, ds.n_rows, ds.n_metrics, means);
    // Compute z-scores directly into each Stock's z array
    for (int i = 0; i < ds.n_rows; i++) {
        for (int j = 0; j < ds.n_metrics; j++) {
            if (stddev[j] != 0.0) {
                ds.stocks[i].z[j] = (data[i][j] - means[j]) / stddev[j];
            } else {
                ds.stocks[i].z[j] = 0.0;
            }
        }
        // Set any unused metric slots (if n_metrics < MAX_METRICS) to 0.0
        for (int j = ds.n_metrics; j < MAX_METRICS; j++) {
            ds.stocks[i].z[j] = 0.0;
        }
    }
    // Free intermediate arrays and buffers
```

```c
    for (int i = 0; i < ds.n_rows; i++) {
        free(data[i]);
        free(missing[i]);
    }
    free(data);
    free(missing);
    free(means);
    free(stddev);
    return ds;
}


// KD-tree + NN-search core (previously referenced via extern, which caused link errors)


int *g_sel = NULL; // global pointer to the currently selected metric indices (set in main)


// Evaluation support (used by Evaluation.c)
static long long g_kd_visit_counter = 0;   // counts KDNodes visited during the last query

void kd_reset_visit_counter(void) {        // call before each NN query
    g_kd_visit_counter = 0;               // reset counter
}

long long kd_get_visit_counter(void) {     // call after each NN query
    return g_kd_visit_counter;            // return number of visited KD nodes
}

// Internal KD-tree node (each node stores one Stock pointer and two subtrees)
typedef struct KDNode {
    Stock *s; // the stock stored at this node (the median on the chosen axis)
    int axis; // which axis (0..k-1) we split on at this depth
    struct KDNode *left; // left subtree (values < median on this axis)
    struct KDNode *right; // right subtree (values > median on this axis)
} KDNode;

static int g_axis = 0; // current axis during qsort comparisons (set just before sorting)

// Compare two Stock* pointers along the current axis (used by qsort in kd_build)
static int stock_ptr_cmp_axis(const void *a, const void *b) {
    const Stock *sa = *(const Stock * const *)a; // dereference to get Stock* (left item)
    const Stock *sb = *(const Stock * const *)b; // dereference to get Stock* (right item)
    const int metric = g_sel ? g_sel[g_axis] : g_axis; // map axis->actual metric index
    const double da = sa->z[metric]; // value of sa on this metric
    const double db = sb->z[metric]; // value of sb on this metric
    if (da < db) return -1; // sa should come first
    if (da > db) return 1;  // sb should come first
    return 0; // equal on this axis
}

// Build a balanced KD-tree from an array of Stock* pointers.
// NOTE: Signature returns Stock** only because main already used that type.
//       Internally we allocate KDNode and cast it to/from Stock** when passing around.
Stock **kd_build(Stock **points, int n, int depth, int k) {
    if (n <= 0) return NULL; // base case: no points => empty tree

    const int axis = (k > 0) ? (depth % k) : 0; // choose split axis based on depth
    g_axis = axis; // tell comparator which axis to use for sorting
    qsort(points, (size_t)n, sizeof(Stock *), stock_ptr_cmp_axis); // sort points by axis

    const int mid = n / 2; // median index

    KDNode *node = (KDNode *)xcalloc(1, sizeof(KDNode)); // allocate one node (zeroed)
    node->s = points[mid]; // store median stock at this node
```

```
    node->axis = axis; // remember which axis we split on here
    node->left = (KDNode *)kd_build(points, mid, depth + 1, k); // build left half
    node->right = (KDNode *)kd_build(points + mid + 1, n - mid - 1, depth + 1, k); // right half

    return (Stock **)node; // return node pointer, but cast to match main's type
}
// Free an entire KD-tree allocated by kd_build().

void kd_free(Stock **root) {
    if (!root) return;                  // empty tree => nothing to free

    KDNode *node = (KDNode *)root;      // reinterpret as KDNode*

    kd_free((Stock **)node->left);      // free left subtree
    kd_free((Stock **)node->right);     // free right subtree

    free(node);                         // free this node
}


// Compute squared Euclidean distance in the selected k-dimensional metric space.
static double dist2_k(const Stock *a, const Stock *b, const int *metric_sel, int k) {
    double acc = 0.0; // accumulator for sum of squared differences
    for (int i = 0; i < k; i++) { // loop across each selected axis
        const int m = metric_sel[i]; // actual metric index in Stock.z[]
        const double d = a->z[m] - b->z[m]; // difference along this metric
        acc += d * d; // add squared difference
    }
    return acc; // return total squared distance
}

// Recursive nearest-neighbour search on the KD-tree.
// NOTE: node is typed as Stock* in main, but it is really a KDNode* under the hood.
const Stock *kd_nearest_neighbor(const Stock *node,
                                 const Stock *target,
                                 const Stock *best,
                                 const int *metric_sel,
                                 int k,
                                 double *best_dist2)
{
    if (!node) return best; // empty subtree => no change
    g_kd_visit_counter++; // count this KDNode visit (for evaluation metrics)

    // If main passed best=target and best_dist2=0, treat it as    uninitialised   .
    if (best == target && best_dist2 && *best_dist2 == 0.0) {
        *best_dist2 = INFINITY; // start with   infinitely   b a d  distance
    }

    const KDNode *n = (const KDNode *)node; // reinterpret node pointer as KDNode*

    // Evaluate this node's stock as a candidate (but skip target itself).
    if (n->s && n->s != target) {
        const double d2 = dist2_k(n->s, target, metric_sel, k); // compute distance^2
        if (!best_dist2 || d2 < *best_dist2) { // if this is better than current best
            best = n->s; // update best stock
            if (best_dist2) *best_dist2 = d2; // update best distance^2
        }
    }

    // Decide which branch is    near    and which is    far    based on the split axis.
    const int axis = n->axis; // axis used at this node (0..k-1)
    const int metric = metric_sel[axis]; // map axis to real metric index
    const double diff = target->z[metric] - n->s->z[metric]; // signed distance to split

```

```
    const KDNode *near_branch = (diff < 0.0) ? n->left : n->right; // go left if target < split
    const KDNode *far_branch  = (diff < 0.0) ? n->right : n->left; // other side

    // Explore the near side first (most likely to contain nearest neighbour).
    best = kd_nearest_neighbor((const Stock *)near_branch, target, best, metric_sel, k, best_dist2);

    // Only explore the far side if the splitting plane could still contain a closer point.
    if (best_dist2 && (diff * diff) < *best_dist2) {
        best = kd_nearest_neighbor((const Stock *)far_branch, target, best, metric_sel, k, best_dist2);
    }

    return best; // return best stock found in this subtree
}

// Parse metric selection text (supports: "all", "1 2 3", "1-4", and mixes of these).
int parse_metric_selection(const char *input, char *metric_names[], int total, int *out_indices) {
    if (!input || !out_indices || total <= 0) return -1; // sanity checks

    // Copy input so we can safely edit/tokenize it.
    char buf[LINE_BUF]; // local scratch buffer
    snprintf(buf, sizeof(buf), "%s", input); // copy input into buf

    // Trim leading spaces.
    char *p = buf; // p will walk across buf
    while (*p && isspace((unsigned char)*p)) p++; // skip whitespace

    // Handle "all" (case-insensitive).
    if (strcasecmp_ascii(p, "all") == 0) { // user selected all metrics
        for (int i = 0; i < total; i++) out_indices[i] = i; // output 0..total-1
        return total; // k = total metrics
    }

    int used[MAX_METRICS] = {0}; // track duplicates so we don't output same metric twice
    int k = 0; // how many metrics we have selected so far

    // Tokenise on spaces and commas (simple but works for typical inputs).
    for (char *tok = strtok(p, " ,\t"); tok; tok = strtok(NULL, " ,\t")) {
        // Range format: a-b
        char *dash = strchr(tok, '-'); // find '-' if it exists
        if (dash) {
            *dash = '\0'; // split into two substrings
            int a = atoi(tok); // parse left bound (1-based)
            int b = atoi(dash + 1); // parse right bound (1-based)
            if (a <= 0 || b <= 0) return -1; // invalid bounds
            if (a > b) { int tmp = a; a = b; b = tmp; } // allow reversed ranges
            if (a < 1 || b > total) return -1; // out of range
            for (int v = a; v <= b; v++) {
                int idx = v - 1; // convert to 0-based index
                if (idx < 0 || idx >= total) continue; // extra safety
                if (!used[idx]) { // only add if not already selected
                    used[idx] = 1; // mark as used
                    out_indices[k++] = idx; // append to output list
                    if (k >= total) break; // cannot select more than total
                }
            }
            continue; // move to next token
        }

        // Single number format: "7"
        bool all_digits = true; // assume digits until proven otherwise
        for (const char *q = tok; *q; q++) {
```

```c
            if (!isdigit((unsigned char)*q)) { all_digits = false; break; } // detect non-digit
        }
        if (all_digits) {
            int v = atoi(tok); // parse 1-based metric number
            if (v < 1 || v > total) return -1; // validate range
            int idx = v - 1; // convert to 0-based
            if (!used[idx]) {
                used[idx] = 1; // mark as used
                out_indices[k++] = idx; // add it
            }
            continue; // next token
        }

        // Optional: allow selecting by metric name (e.g., "PE" or "Market Cap").
        // This is best-effort: if it doesn't match, we reject.
        int found = -1; // index of matched metric
        for (int i = 0; i < total; i++) {
            if (metric_names && metric_names[i] && strcasecmp_ascii(metric_names[i], tok) == 0) {
                found = i; // metric matched by name
                break;
            }
        }
        if (found >= 0 && found < total) {
            if (!used[found]) {
                used[found] = 1; // mark used
                out_indices[k++] = found; // add index
            }
        } else {
            return -1; // unknown token
        }
    }

    return (k > 0) ? k : -1; // must select at least one metric
}

// Find a stock by exact ticker match, or exact name match (case-insensitive).
const Stock *find_stock(const Stock *stocks, int n, const char *query) {
    if (!stocks || n <= 0 || !query) return NULL; // sanity checks

    // Copy + trim query (users often include spaces/newlines)
    char qbuf[512]; // query buffer
    snprintf(qbuf, sizeof(qbuf), "%s", query); // copy query
    char *q = qbuf; // pointer to start
    while (*q && isspace((unsigned char)*q)) q++; // trim leading spaces
    char *end = q + strlen(q); // pointer to end
    while (end > q && isspace((unsigned char)end[-1])) *--end = '\0'; // trim trailing spaces

    // 1) Prefer exact ticker match.
    for (int i = 0; i < n; i++) {
        if (stocks[i].ticker && strcasecmp_ascii(stocks[i].ticker, q) == 0) {
            return &stocks[i]; // found by ticker
        }
    }

    // 2) Exact company name match.
    for (int i = 0; i < n; i++) {
        if (stocks[i].name && strcasecmp_ascii(stocks[i].name, q) == 0) {
            return &stocks[i]; // found by name
        }
    }

    return NULL; // not found
```

```c
}

int main(int argc, char **argv) {
    if (argc < 2) {
        fprintf(stderr, "Usage: %s <csv_file>\n", argv[0]);
        return 1;
    }
    // Load and standardize dataset from the given CSV file
    Dataset ds = load_dataset_and_standardise_via_other_file(argv[1]);

    /* Print available metrics (helpful context for user) */
    printf("Available metrics (%d):\n", ds.n_metrics);
    for (int j = 0; j < ds.n_metrics; j++) {
        printf("  %2d) %s\n", j + 1, ds.metric_names[j] ? ds.metric_names[j] : "Metric");
    }

    /* Prompt 1 (exact text as requested) */
    char line1[LINE_BUF];
    printf("Which metrics would you like to consider:");
    if (!fgets(line1, sizeof(line1), stdin)) {
        dataset_free(&ds);
        fprintf(stderr, "\nError: failed to read input.\n");
        return 1;
    }
    chomp(line1);
    // Parse selected metrics (could be "all" or numeric indices)
    int sel[MAX_METRICS];
    extern int parse_metric_selection(const char *input, char *metric_names[], int total,
        int *out_indices);
    int k = parse_metric_selection(line1, ds.metric_names, ds.n_metrics, sel);
    if (k < 0) {
        dataset_free(&ds);
        fprintf(stderr, "Error: could not parse metric selection. Use e.g. 'all' or '1 2 3
            4' or '1-4'.\n");
        return 1;
    }

    // Make selection visible to KD comparator (global pointer used in kd_nearest)
    extern int *g_sel;
    g_sel = sel;

    /* Prompt 2 (exact text as requested) */
    char line2[LINE_BUF];
    printf("Which stock would like to conduct a NN-search for with these metrics:");
    if (!fgets(line2, sizeof(line2), stdin)) {
        dataset_free(&ds);
        fprintf(stderr, "\nError: failed to read input.\n");
        return 1;
    }
    chomp(line2);

    extern const Stock* find_stock(const Stock *stocks, int n, const char *query);
    const Stock *query = find_stock(ds.stocks, ds.n_rows, line2);
    if (!query) {
        dataset_free(&ds);
        fprintf(stderr, "Error: stock not found by ticker or company name.\n");
        return 1;
    }

    /* Build kd-tree in k dimensions (selected metrics only) */
    extern Stock** kd_build(Stock **points, int n, int depth, int k);
    Stock **ptrs = (Stock **) xmalloc((size_t) ds.n_rows * sizeof(Stock *));
    for (int i = 0; i < ds.n_rows; i++) {
        ptrs[i] = &ds.stocks[i];
    }
```

```c
    // Recursively build k-d tree
    Stock **root = kd_build(ptrs, ds.n_rows, 0, k);

    /* Nearest neighbour search (exclude the query stock itself) */
    extern const Stock* kd_nearest_neighbor(const Stock *node, const Stock *target, const Stock *best, const int *metric_sel, int k, double *best_dist2);
    double best_d2 = 0.0;
    const Stock *nn = kd_nearest_neighbor((const Stock*)root, query, query, sel, k, &best_d2);

    // Output the query and nearest neighbor result
    printf("\nQuery: %s (%s)\n", query->name, query->ticker);
    printf("Metrics used (%d):\n", k);
    for (int i = 0; i < k; i++) {
        int j = sel[i];
        printf("  - %s\n", ds.metric_names[j] ? ds.metric_names[j] : "Metric");
    }
    if (!nn) {
        printf("\nNo nearest neighbor found (only one stock in dataset).\n");
    } else {
        printf("\nNearest neighbor: %s (%s)\n", nn->name, nn->ticker);
        /* Transparency: show per-metric z-scores + abs diffs on the selected metrics */
        printf("Distance (Euclidean) in %d-dimensional metric space: %.4f\n", k, sqrt(best_d2));
        printf("\nPer-metric z-scores (query vs neighbour) and |diff| (z-score units):\n");
    for (int i = 0; i < k; i++) { // loop over selected metrics
    int j = sel[i]; // actual metric index in the full metric list
    double qz = query->z[j]; // query stock z-score for this metric
    double nz = nn->z[j]; // neighbour stock z-score for this metric
    double diff = fabs(qz - nz); // absolute difference in z-score units
    printf("  %s: query=%g, nn=%g, |diff|=%g\n",
           ds.metric_names[j] ? ds.metric_names[j] : "Metric", qz, nz, diff); // print a readable breakdown
}
    }
    // Clean up
    dataset_free(&ds);
    free(ptrs);
    return 0;
}
```

Summary:

- Reads a CSV containing (ticker, company name, metrics...) (header optional).
- Detects which column is ticker/name.
- Reads up to 12 numeric metrics per row, marking missing values.
- Calls external standardisation functions (means/std/z-scores) to compute z-scores.
- Prompts you to choose which metrics to use (subset of dimensions).
- Prompts you to choose a stock by ticker or name.
- Builds a k-d tree in k = number of selected metrics dimensions.
- Runs a Euclidean nearest-neighbour search (excluding the query stock itself).
- Prints the nearest neighbour and per-metric z-score differences

Compile instructions: to run NN-Search

- gcc -DBUILD_AS_LIBRARY -c CSV_Parsing_and_Standardisation.c -o Standardise_no_main.o
- gcc Build_KD-Tree_and_NNSearch.c Standardise_no_main.o -o Run_program -lm
- ./Run_program Data_for_Project.csv

Note '-DBUILD_AS_LIBRARY' has to be included as there are two mains.

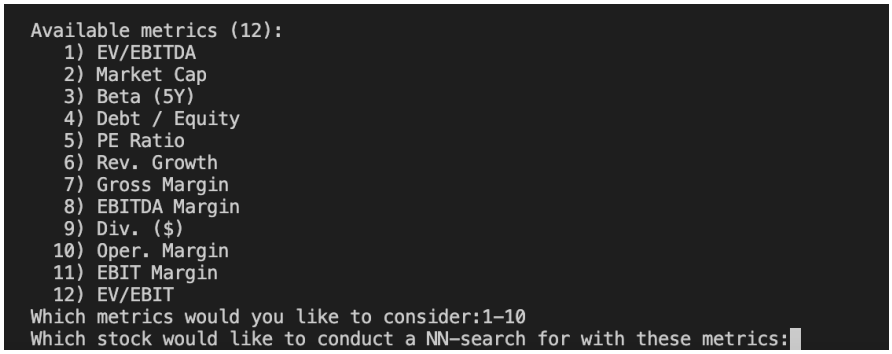


Figure 1: Prompt when program is run.

5 Performance Evaluation

5.1 Outline of naive searches

I chose a naive (brute-force) nearest neighbour search for baseline comparator. For a given query stock, the algorithm computes the Euclidean distance between the query point and every other stock in the dataset, then returns the stock with the smallest distance.

Algorithmically, the naive search is a simple linear scan: initialise the best distance as infinity; loop through all N stocks; compute the squared Euclidean distance in k dimensions; update the current best if a smaller distance is found; and finally return the best candidate. This guarantees correctness because it explicitly checks every possible neighbour. However, the computational cost scales linearly with dataset size: each query requires $N - 1$ distance evaluations, and each distance evaluation costs $O(k)$. Therefore, the time complexity per query is $O(Nk)$. This naive approach is used as the ground truth and performance baseline against which the KD-tree implementation is evaluated.

5.2 Implementation and comparison

To evaluate performance, I wrote an evaluation driver (`Evaluation_Final.c`) that loads the CSV dataset, standardises the selected metrics, and then benchmarks two exact nearest neighbour methods on the same query set: (i) naive linear scan and (ii) KD-tree nearest neighbour search. For fairness, both methods use the same distance definition (Euclidean distance on standardised features), and both exclude the query stock itself from being returned as its own nearest neighbour.

For each dimensionality k, the evaluation program performs three core steps. First, it builds a KD-tree using the first k metrics and records the one-off build time. Second, it runs the naive nearest neighbour search for the chosen number of queries Q (in my reported run, Q was set to cover all stocks so that results reflect full-dataset behaviour). Third, it runs the KD-tree nearest neighbour search for the same Q queries and records the total query time. From these totals it computes a speedup factor

$$\text{speedup} = \frac{\text{naive(ms)}}{\text{kd(ms)}}.$$

In addition to runtime, the program records tree-search effort by counting how many KD-tree nodes are visited per query and estimating the percentage of the dataset pruned as

$$100 \cdot \left(1 - \frac{\text{avgVisited}}{N}\right).$$

A correctness check is also included by comparing KD-tree answers against naive answers; mismatches were 0 across all tested values of k, indicating the KD-tree implementation is returning exact nearest neighbours.

```
Results (lower time is better)
-------------------------------------------------------------------------------------------------------------
-----
 k |    build(ms)    |   naive(ms)   |   kd(ms)   |  speedup |   avg visited nodes | avg pruned (%)  | mismat
ches
-------------------------------------------------------------------------------------------------------------
-----
|  1 |        1.681 |    62.657000 |  1.308000 |   47.903 |              11.64 |         99.42 |          0 |
|  2 |        2.433 |    81.661000 |  2.897000 |   28.188 |              28.77 |         98.56 |          0 |
|  3 |        2.349 |   104.109000 |  8.923000 |   11.667 |              88.20 |         95.60 |          0 |
|  4 |        2.320 |   126.048000 | 17.605000 |    7.160 |             154.73 |         92.28 |          0 |
|  5 |        4.473 |   179.063000 | 32.288000 |    5.546 |             199.74 |         90.03 |          0 |
|  6 |        2.375 |   189.281000 | 40.266000 |    4.701 |             274.74 |         86.29 |          0 |
|  7 |        2.744 |   220.693000 | 69.713000 |    3.166 |             383.49 |         80.86 |          0 |
|  8 |        2.578 |   262.958000 | 84.330000 |    3.118 |             497.20 |         75.19 |          0 |
|  9 |        2.781 |   255.357000 | 97.491000 |    2.619 |             548.38 |         72.64 |          0 |
| 10 |        2.314 |   305.909000 | 98.665000 |    3.100 |             556.58 |         72.23 |          0 |
| 11 |        2.309 |   308.578000 | 109.424000 |    2.820 |             573.71 |         71.37 |          0
|
| 12 |        2.339 |   345.115000 | 127.366000 |    2.710 |             613.69 |         69.38 |          0
|
-------------------------------------------------------------------------------------------------------------
-----
```

Figure 2: Evaluation output table for $k = 1$ through $k = 12$, including build time, naive time, KD-tree time, speedup, average visited nodes, average pruned percentage, and mismatches.

The results show that the KD-tree is consistently faster than naive search across all tested dimensions, but the magnitude of the advantage depends heavily on k. In low dimensions, pruning is extremely effective and the KD-tree achieves very large speedups: for example at $k = 1$, KD-tree query time is ~ 1.3 ms versus ~ 62.657 ms for naive search, a speedup of $\sim 48\times$, with an average of only ~ 11.64 nodes visited per query (99.42% pruned). As the number of dimensions increases, KD-tree query time rises and the speedup compresses: at $k = 12$, KD-tree time is ~ 127.366 ms versus ~ 345.115 ms naive, giving a speedup of $\sim 2.710\times$, with ~ 613.69 nodes visited per query and pruning dropping to $\sim 69.38\%$. This pattern is central to the discussion of the curse of dimensionality in the next subsection.

For each k, I run Q nearest-neighbour queries and report the total wall-clock query time for naive and KD-tree search (ms). Because both totals are over the same Q, the reported speedup naive/kd is equivalent to the ratio of per-query average times. In the main experiment I set Q = N (all stocks), so results are averaged across the entire dataset's query distribution

My code is below: Evaluation_Final.c

```c
#include <stdio.h>    // printf, fprintf, FILE, fopen, fgets
#include <stdlib.h>   // malloc, calloc, free, exit, atoi, strtoul
#include <string.h>   // strlen, memcpy, memmove, strtok, strchr, snprintf
#include <stdbool.h>  // bool, true, false
#include <ctype.h>    // isspace, isdigit, tolower
#include <math.h>     // fabs, sqrt, INFINITY
#include <time.h>     // clock_gettime, timespec

```

```c


#define MAX_K    12         // maximum dimensions used in evaluation
#define LINE_BUF 16384      // large line buffer for robust CSV parsing

/*
   External standardisation functions (from your other file)
*/
extern double*  computeColumnMeans(double **data, bool **missing, int rows, int cols);
               // means per column
extern double*  computeColumnStd(double **data, int rows, int cols, double *means);
                // std dev per column
extern double** computeZScores(double **data, int rows, int cols, double *means, double
    *stddev); // z-scores

/*
   Stock + Dataset representation
 */
typedef struct {
    char  *name;                // company name (string)
    char  *ticker;              // ticker symbol (string)
    double z[MAX_K];            // standardized metrics (z-scores)
} Stock;

typedef struct {
    Stock *stocks;              // array of stocks
    int    n;                   // number of rows (stocks)
    int    m;                   // number of metrics loaded (<= 12)
    char  *metric_names[MAX_K]; // metric names (from CSV header)
} Dataset;

/*
   External KD-tree API (from Build_KD-Tree_NNSearch.c)
*/
extern int *g_sel;  // KD-tree module uses this to map axis -> selected metric index

extern Stock **kd_build(Stock **points, int n, int depth, int k); // build KD-tree (returns
    opaque root)
extern const Stock *kd_nearest_neighbor(                          // exact NN search
    const Stock *node,
    const Stock *target,
    const Stock *best,
    const int   *metric_sel,
    int          k,
    double      *best_dist2
);

extern void kd_free(Stock **root);                  // frees KD-tree nodes
extern void kd_reset_visit_counter(void);           // resets visited nodes counter
extern long long kd_get_visit_counter(void);        // gets visited nodes counter

/*
   Timing helpers
*/
static inline long long now_ns(void) {  // current time in nanoseconds
    struct timespec ts;                 // timespec struct
    clock_gettime(CLOCK_MONOTONIC, &ts);      // monotonic clock (benchmark-safe)
    return (long long)ts.tv_sec * 1000000000LL + ts.tv_nsec;   // convert to ns
}

static inline double ns_to_ms(long long ns) { // convert ns to ms
    return (double)ns / 1e6;   // ms = ns / 1e6
}

/*
```

```c
   Safe memory helpers
*/
static void *xmalloc(size_t n) {                  // malloc with failure check
    void *p = malloc(n);                            // allocate
    if (!p) {                                       // if allocation failed
        fprintf(stderr, "Out of memory.\n");        // error message
        exit(2);                                    // abort
    }
    return p;                                       // return pointer
}

static void *xcalloc(size_t n, size_t s) {  // calloc with failure check
    void *p = calloc(n, s);     // allocate + zero
    if (!p) {                                                   // if allocation failed
        fprintf(stderr, "Out of memory.\n");                    // error message
        exit(2);                                                // abort
    }
    return p;                                                   // return pointer
}

static char *xstrdup(const char *s) {                           // duplicate string safely
    size_t n = strlen(s) + 1;                                   // bytes including '\0'
    char *p = (char*)xmalloc(n);                                // allocate
    memcpy(p, s, n);                                            // copy bytes
    return p;                                                   // return new string
}

/*
   Small string helpers
*/
static void chomp(char *s) {                                    // remove trailing newline(s)
    size_t n = strlen(s);                                       // length
    while (n > 0 && (s[n-1] == '\n' || s[n-1] == '\r')) {        // while last is newline/cr
        s[--n] = '\0';                                          // delete it
    }
}

static char *trim(char *s) {                                    // trim leading/trailing spaces
    while (*s && isspace((unsigned char)*s)) s++;               // skip leading spaces
    if (*s == '\0') return s;                                   // all spaces -> empty
    char *e = s + strlen(s) - 1;                                // end pointer
    while (e > s && isspace((unsigned char)*e)) *e-- = '\0';    // trim trailing spaces
    return s;                                                   // return trimmed string
}

static void strip_utf8_bom(char *s) {                           // remove UTF-8 BOM if present
    if (!s) return;                                             // null guard
    unsigned char *u = (unsigned char*)s;                       // byte-wise pointer
    if (u[0] == 0xEF && u[1] == 0xBB && u[2] == 0xBF) {          // BOM pattern
        memmove(s, s + 3, strlen(s + 3) + 1);                   // shift left by 3 bytes
    }
}

static int strcasecmp_ascii(const char *a, const char *b) {     // ASCII-only case-insensitive compare
    while (*a && *b) {                                          // while both have chars
        unsigned char ca = (unsigned char)tolower((unsigned char)*a); // lower(a)
        unsigned char cb = (unsigned char)tolower((unsigned char)*b); // lower(b)
        if (ca != cb) return (int)ca - (int)cb;                 // difference
        a++;                                                    // advance
```

```c
        b++;                                                        // advance
    }
    return (int)(unsigned char)tolower((unsigned char)*a) -         // compare remaining
           (int)(unsigned char)tolower((unsigned char)*b);
}

/*
   Robust CSV splitting (handles quoted fields with commas)
 */
static int split_csv_inplace(char *line, char **fields, int max_fields) {
    int n = 0;   // number of fields found
    char *p = line;    // read cursor
    char *w = line;     // write cursor (in-place rewrite)

    while (*p && n < max_fields) {  // while data remains and room
        fields[n++] = w;  // start of this field

        if (*p == '"') {  // quoted field?
            p++;     // skip opening quote
            while (*p) {                                            // read until closing quote
                if (*p == '"') {                                    // quote found
                    if (p[1] == '"') {                              // escaped quote ("")
                        *w++ = '"';                                 // write one quote
                        p += 2;                                     // skip both
                        continue;                                   // continue copying
                    }
                    p++;                                            // consume closing quote
                    break;                                          // stop quoted field
                }
                *w++ = *p++;                                        // copy character
            }
            while (*p && *p != ',') p++;                            // consume until comma/end
        } else {                                                    // unquoted field
            while (*p && *p != ',') {                               // copy until comma/end
                *w++ = *p++;                                        // copy character
            }
        }

        bool has_comma = (*p == ',');                               // delimiter present?
        *w++ = '\0';                                                // terminate this field
        if (has_comma) p++;                                         // skip comma
    }

    for (int i = 0; i < n; i++) fields[i] = trim(fields[i]);        // trim each field
    if (n > 0) strip_utf8_bom(fields[0]);                           // strip BOM from first field
    return n;                                                       // return field count
}

/*
   Numeric parsing (accepts commas and optional trailing %)
 */
static bool parse_double_loose(const char *s, double *out) {
    if (!s) return false;                                           // null guard
    while (*s && isspace((unsigned char)*s)) s++;                   // skip leading spaces
    if (*s == '\0') return false;                                   // empty -> fail

    char buf[256];                                                  // scratch buffer
    size_t L = strlen(s);                                           // input length
    if (L >= sizeof(buf)) L = sizeof(buf) - 1;                      // clamp
    memcpy(buf, s, L);                                              // copy
    buf[L] = '\0';                                                  // terminate

```

```c
    char *t = trim(buf);                                        // trim in-place
    if (*t == '\0') return false;                               // empty after trim -> fail

    if (*t && !isdigit((unsigned char)*t) && *t != '-' && *t != '+' && *t != '.') {
        t++;                                                    // skip one leading symbol (e.g., '$')
    }

    char cleaned[256];                                          // cleaned numeric string
    size_t ci = 0;                                              // cleaned index
    for (size_t i = 0; t[i] && ci + 1 < sizeof(cleaned); i++) {     // walk chars
        if (t[i] == ',') continue;                              // remove thousands separators
        cleaned[ci++] = t[i];                                   // copy char
    }
    cleaned[ci] = '\0';                                         // terminate

    char *end = NULL;                                           // strtod end pointer
    double v = strtod(cleaned, &end);                           // convert
    if (end == cleaned) return false;                           // no conversion

    while (*end && isspace((unsigned char)*end)) end++;         // skip trailing spaces
    if (*end == '%') {                                          // allow trailing %
        end++;                                                  // consume %
        while (*end && isspace((unsigned char)*end)) end++;     // skip spaces after %
    }
    if (*end != '\0') return false;                             // any garbage -> fail

    *out = v;                                                   // success output
    return true;                                                // success
}

/*
   Dataset cleanup
 */
static void dataset_free(Dataset *ds) {
    if (!ds) return;                                            // null guard

    if (ds->stocks) {                                           // if stock array exists
        for (int i = 0; i < ds->n; i++) {                       // free each stock strings
            free(ds->stocks[i].name);                           // free name
            free(ds->stocks[i].ticker);                         // free ticker
        }
        free(ds->stocks);                                       // free stock array
    }

    for (int j = 0; j < MAX_K; j++) {                            // free metric names
        free(ds->metric_names[j]);                              // free each
        ds->metric_names[j] = NULL;                             // avoid dangling pointer
    }

    memset(ds, 0, sizeof(*ds));                                 // zero the struct
}

/*
   Load CSV + standardise into z-scores using linked functions
 */
static Dataset load_and_standardise(const char *filename) {
    Dataset ds;                                                 // dataset result
    memset(&ds, 0, sizeof(ds));                                 // init to zeros

    FILE *fp = fopen(filename, "r");                            // open CSV
    if (!fp) {                                                  // failed?
```

```c
        fprintf(stderr, "Error: could not open '%s'\n", filename);  // error
        exit(1);                                                    // abort
    }

    char line[LINE_BUF];                                            // line buffer
    char *fields[256];                                              // field pointers (in-place)

    int header_nf = 0;                                              // number of header fields

    while (fgets(line, sizeof(line), fp)) {                         // read line
        chomp(line);                                                // remove newline
        if (line[0] == '\0') continue;                              // skip empty
        header_nf = split_csv_inplace(line, fields, 256);           // split header
        if (header_nf >= 3) break;                                  // need at least 2 id cols + 1 metric
    }

    if (header_nf < 3) {                                            // header invalid?
        fclose(fp);                                                 // close file
        fprintf(stderr, "Error: CSV header invalid.\n");             // error
        exit(1);                                                    // abort
    }

    int m = header_nf - 2;                                          // metrics count = total - 2
    if (m > MAX_K) m = MAX_K;                                       // clamp to 12
    if (m <= 0) {                                                   // no metrics?
        fclose(fp);                                                 // close
        fprintf(stderr, "Error: no numeric metric columns detected.\n"); // error
        exit(1);                                                    // abort
    }

    ds.m = m;                                                       // store number of metrics

    for (int j = 0; j < m; j++) {                                   // copy metric names from header
        ds.metric_names[j] = xstrdup(fields[2 + j]);                // allocate + copy
    }

    long pos_after_header = ftell(fp);                              // remember file pos after header

    int rows = 0;                                                   // count rows
    while (fgets(line, sizeof(line), fp)) {                         // read remaining lines
        chomp(line);                                                // strip newline
        if (line[0] == '\0') continue;                              // skip empty
        rows++;                                                     // count row
    }

    if (rows <= 0) {                                                // no data rows?
        fclose(fp);                                                 // close
        fprintf(stderr, "Error: no data rows found.\n");             // error
        exit(1);                                                    // abort
    }

    ds.n = rows;                                                    // initial row count
    ds.stocks = (Stock*)xcalloc((size_t)rows, sizeof(Stock));       // allocate stocks array

    double **data = (double**)xmalloc((size_t)rows * sizeof(double*)); // raw numeric values
    bool   **missing = (bool**)xmalloc((size_t)rows * sizeof(bool*));  // missing flags

    for (int i = 0; i < rows; i++) {                                // allocate per-row
```

```
        arrays
        data[i] = (double*)xmalloc((size_t)m * sizeof(double));     // m doubles
        missing[i] = (bool*)xmalloc((size_t)m * sizeof(bool));      // m bools
    }

    fseek(fp, pos_after_header, SEEK_SET);                          // rewind to first data
        row

    int r = 0;                                                      // actual rows parsed
    while (r < rows && fgets(line, sizeof(line), fp)) {             // read data lines
        chomp(line);                                                // remove newline
        if (line[0] == '\0') continue;                              // skip empty

        int nf = split_csv_inplace(line, fields, 256);              // split fields
        if (nf < 2) continue;                                       // must have ticker+name
            at least

        ds.stocks[r].ticker = xstrdup(fields[0]);                   // column 1: ticker
        ds.stocks[r].name   = xstrdup(fields[1]);                   // column 2: company name

        for (int j = 0; j < m; j++) {                               // parse metrics columns
            double v = 0.0;                                         // parsed value
            bool ok = (2 + j < nf) && parse_double_loose(fields[2 + j], &v); // parse if
                present
            data[r][j] = ok ? v : 0.0;                              // store (0 if missing)
            missing[r][j] = !ok;                                    // mark missing
        }

        r++;                                                        // next row
    }

    fclose(fp);                                                     // close file

    ds.n = r;                                                       // set actual row count
        (if some skipped)

    double *means = computeColumnMeans(data, missing, ds.n, m);     // compute means (ignore
        missing)

    for (int i = 0; i < ds.n; i++) {                                // mean-impute missing
        values
        for (int j = 0; j < m; j++) {                               // per metric
            if (missing[i][j]) data[i][j] = means[j];               // impute
        }
    }

    double *stddev = computeColumnStd(data, ds.n, m, means);        // compute standard
        deviations
    double **z = computeZScores(data, ds.n, m, means, stddev);      // compute z-scores

    for (int i = 0; i < ds.n; i++) {                                // copy z-scores into
        Stock.z[]
        for (int j = 0; j < m; j++) ds.stocks[i].z[j] = z[i][j];    // copy used metrics
        for (int j = m; j < MAX_K; j++) ds.stocks[i].z[j] = 0.0;    // zero unused slots
    }

    for (int i = 0; i < rows; i++) {                                // free raw arrays
        free(data[i]);                                              // free row
        free(missing[i]);                                           // free row flags
        if (i < ds.n) free(z[i]);                                   // free z row (only
            allocated for parsed rows)
    }

    free(data);                                                     // free row pointers
    free(missing);                                                  // free row pointers
```

```c
    free(z);                                                    // free z pointers
    free(means);                                                // free means
    free(stddev);                                               // free stddev

    return ds;                                                  // return populated dataset
}

/*
   Naive nearest neighbour (linear scan)
 */
static inline double dist2_sel(const double *a, const double *b, const int *sel, int k) {
    double d2 = 0.0;                                            // squared distance accumulator
    for (int i = 0; i < k; i++) {                               // loop over selected metrics
        int j = sel ? sel[i] : i;                               // map axis->metric index
        double diff = a[j] - b[j];                              // difference in this metric
        d2 += diff * diff;                                      // add square
    }
    return d2;                                                  // return squared distance
}

static int naive_nn_index(const Stock *stocks, int n, int q_idx, const int *sel, int k, double *out_d2) {
    const double *q = stocks[q_idx].z;                          // query point
    int best = -1;                                              // best index found
    double best_d2 = INFINITY;                                  // best distance so far

    for (int i = 0; i < n; i++) {                               // scan all points
        if (i == q_idx) continue;                               // skip the query itself
        double d2 = dist2_sel(stocks[i].z, q, sel, k);          // compute squared distance
        if (d2 < best_d2) {                                     // improvement?
            best_d2 = d2;                                       // update best distance
            best = i;                                           // update best index
        }
    }

    if (out_d2) *out_d2 = best_d2;                              // output squared distance if requested
    return best;                                                // return nearest index
}

/*
   Query selection helper (shuffle indices)
 */
static void shuffle_ints(int *a, int n, unsigned int seed) {
    srand(seed);                                                // seed RNG
    for (int i = n - 1; i > 0; i--) {
        int j = rand() % (i + 1);                                // pick random 0..i
        int tmp = a[i];                                         // swap
        a[i] = a[j];
        a[j] = tmp;
    }
}

/*
   Prompt helpers
 */
static bool read_line(const char *prompt, char *buf, size_t bufsz) {
    printf("%s", prompt);                                       // print prompt
    fflush(stdout);                                             // flush output
```

```c
    if (!fgets(buf, bufsz, stdin)) return false;                    // read line (fail if EOF)
    chomp(buf);                                                     // strip newline
    return true;                                                    // success
}

static bool is_all_word(const char *s) {
    while (*s && isspace((unsigned char)*s)) s++;                   // skip leading spaces
    if (*s == '\0') return false;                                   // empty -> false
    return strcasecmp_ascii(s, "all") == 0;                         // equals "all" ignoring case?
}

/* Parse k list input: \"8\", \"8 12\", \"1-12\", \"all\".
   Returns number of k values; fills ks_out[]. */
static int parse_k_list(const char *in, int *ks_out) {
    bool used[MAX_K + 1] = {0};                                     // track which k selected
    int count = 0;                                                  // number of ks output

    char tmp[256];                                                  // scratch
    snprintf(tmp, sizeof(tmp), "%s", in);                            // copy
    for (char *p = tmp; *p; p++) {                                  // normalize separators
        if (*p == ',' || *p == ';') *p = ' ';                       // replace with spaces
    }

    char *s = trim(tmp);                                            // trim copy

    if (strcasecmp_ascii(s, "all") == 0) {                          // all dimensions?
        for (int k = 1; k <= MAX_K; k++) used[k] = true;            // mark all
    } else {
        char *tok = strtok(s, " \t");                               // tokenize on whitespace
        while (tok) {
            char *dash = strchr(tok, '-');                          // look for range
            if (dash && dash != tok && dash[1] != '\0') {           // valid range form a-b
                *dash = '\0';                                         // split string
                int a = atoi(tok);                                  // parse left
                int b = atoi(dash + 1);                              // parse right
                if (a < 1) a = 1;                                   // clamp
                if (b > MAX_K) b = MAX_K;                           // clamp
                if (a <= b) {                                       // increasing
                    for (int k = a; k <= b; k++) used[k] = true;    // mark
                } else {                                            // decreasing range allowed
                    for (int k = a; k >= b; k--) used[k] = true;    // mark
                }
            } else {
                int k = atoi(tok);                                  // single number
                if (k >= 1 && k <= MAX_K) used[k] = true;           // mark valid
            }
            tok = strtok(NULL, " \t");                              // next token
        }
    }

    for (int k = 1; k <= MAX_K; k++) {                               // output in ascending order
        if (used[k]) ks_out[count++] = k;                           // append
    }
    return count;                                                    // return how many
}

/*
   Benchmark runner
*/
typedef struct {
    int    k;                 // dimensionality
```

```
    double build_ms;          // KD build time
    double naive_ms;          // naive total query time
    double kd_ms;             // KD total query time
    double speedup;           // naive_ms / kd_ms
    double avg_visited;       // average KD nodes visited per query
    double avg_pruned_pct;    // estimated prune percentage
    int    mismatches;        // correctness mismatches vs naive (optional)
} BenchResult;

static BenchResult run_benchmark_for_k(const Dataset *ds, const int *query_idx, int Q, int k, bool do_compare) {
    BenchResult br;                                                 // benchmark result
    memset(&br, 0, sizeof(br));                                     // init to zero
    br.k = k;                                                       // store k

    int sel[MAX_K];                                                 // selection array (metric indices)
    for (int i = 0; i < k; i++) sel[i] = i;                         // choose first k metrics

    g_sel = sel;                                                    // tell KD module how to map axes -> metrics

    Stock **pts = (Stock**)xmalloc((size_t)ds->n * sizeof(Stock*)); // array of pointers for KD build
    for (int i = 0; i < ds->n; i++) pts[i] = &ds->stocks[i];        // fill pointers

    long long t0 = now_ns();                                        // start timer
    Stock **root = kd_build(pts, ds->n, 0, k);                      // build KD-tree (opaque root)
    long long t1 = now_ns();                                        // end timer
    br.build_ms = ns_to_ms(t1 - t0);                                // store build time

    double d2_tmp = 0.0;                                            // temp distance holder

    t0 = now_ns();                                                  // start naive timer
    for (int qi = 0; qi < Q; qi++) {                                // for each query
        (void)naive_nn_index(ds->stocks, ds->n, query_idx[qi], sel, k, &d2_tmp); // naive NN
    }
    t1 = now_ns();                                                  // stop naive timer
    br.naive_ms = ns_to_ms(t1 - t0);                                // store naive time

    long long visited_total = 0;                                    // sum visited nodes

    t0 = now_ns();                                                  // start KD timer
    for (int qi = 0; qi < Q; qi++) {                                // for each query
        int q = query_idx[qi];                                      // query index
        const Stock *target = &ds->stocks[q];                       // query stock pointer

        kd_reset_visit_counter();                                   // reset visited nodes counter
        double best_d2 = 0.0;                                       // KD module treats 0 with best==target as "uninitialised"

        (void)kd_nearest_neighbor((const Stock*)root, target, target, sel, k, &best_d2); // KD NN search

        visited_total += kd_get_visit_counter();                    // accumulate nodes visited
    }
    t1 = now_ns();                                                  // stop KD timer
    br.kd_ms = ns_to_ms(t1 - t0);                                   // store KD time

    br.speedup = (br.kd_ms > 0.0) ? (br.naive_ms / br.kd_ms) : 0.0; // compute speedup safely
```

```c

    br.avg_visited = (double)visited_total / (double)Q;             // average visited nodes per query
    br.avg_pruned_pct = 100.0 * (1.0 - (br.avg_visited / (double)ds->n)); // prune estimate
    if (br.avg_pruned_pct < 0.0) br.avg_pruned_pct = 0.0;           // clamp at 0

    if (do_compare) {                                               // correctness check enabled?
        const double tol = 1e-9;                                    // tolerance for tie distances
        int mism = 0;                                               // mismatch counter

        for (int qi = 0; qi < Q; qi++) {                            // for each query
            int q = query_idx[qi];                                  // query index

            double d2_naive = 0.0;                                  // naive distance^2
            double d2_kd = 0.0;                                     // kd distance^2

            int nn1 = naive_nn_index(ds->stocks, ds->n, q, sel, k, &d2_naive); // naive result

            const Stock *target = &ds->stocks[q];                   // query pointer
            kd_reset_visit_counter();                               // reset counter (not required for correctness, but consistent)
            d2_kd = 0.0;                                            // init per KD module convention
            const Stock *nnptr = kd_nearest_neighbor((const Stock*)root, target, target, sel, k, &d2_kd); // KD result

            int nn2 = (nnptr != NULL) ? (int)(nnptr - ds->stocks) : -1; // pointer -> index (O(1))

            if (nn1 != nn2) {                                       // different neighbour indices?
                if (fabs(d2_naive - d2_kd) > tol) mism++;           // count mismatch only if distance differs (handles ties)
            }
        }

        br.mismatches = mism;                                       // store mismatch count
    }

    kd_free(root);                                                  // free KD-tree nodes
    free(pts);                                                      // free pointer array used for building

    return br;                                                      // return benchmark result
}

/*
   Printing results
*/
static void print_results_header(void) {
    printf("\nResults (lower time is better)\n");                // headline
    printf("-------------------
```

```
            br->k,           // k
            br->build_ms,     // build time
            br->naive_ms,    // naive time
            br->kd_ms,       // kd time
            br->speedup,     // speedup
            br->avg_visited, // avg visited nodes
            br->avg_pruned_pct, // pruned %
            br->mismatches);     // mismatches
}

/*
   Main
 */
int main(int argc, char **argv) {
    if (argc < 2) {                                                 // require CSV argument
        fprintf(stderr, "Usage: %s <csv_file>\n", argv[0]);        // usage
        return 1;                                                   // error code
    }

    Dataset ds = load_and_standardise(argv[1]);                     // load + standardise dataset
    if (ds.m < 1) {                                                 // no metrics?
        dataset_free(&ds);                                          // cleanup
        fprintf(stderr, "Error: dataset has no metrics.\n");       // error
        return 1;                                                   // abort
    }

    printf("Dataset loaded: N=%d stocks, M=%d metrics available (using first min(M,12)).\n", ds.n, ds.m); // summary
    printf("Metrics (1..%d):\n", ds.m);                            // metrics list header
    for (int j = 0; j < ds.m; j++) {                                // print metric names
        printf("  %2d) %s\n", j + 1, ds.metric_names[j] ? ds.metric_names[j] : "(metric)"); // safe print
    }

    char buf[256];                                                  // input buffer

    if (!read_line("\nHow many queries should be benchmarked? Enter 'all' or a number (e.g., 200): ", buf, sizeof(buf))) {
        dataset_free(&ds);                                          // cleanup
        return 1;                                                   // abort
    }

    int Q = 0;                                                      // number of queries
    bool use_all = is_all_word(buf);                                // did user enter "all"?
    if (use_all) {                                                  // all queries
        Q = ds.n;                                                   // Q = N
    } else {
        Q = atoi(buf);                                              // parse integer
        if (Q <= 0) {                                               // invalid?
            fprintf(stderr, "Invalid query count.\n");             // error
            dataset_free(&ds);                                      // cleanup
            return 1;                                               // abort
        }
        if (Q > ds.n) Q = ds.n;                                     // clamp to N
    }

    unsigned int seed = 42;                                         // default seed
    if (!use_all) {                                                 // only need seed if sampling
        if (!read_line("Random seed for query sampling (press Enter for default 42): ", buf, sizeof(buf))) {
            dataset_free(&ds);                                      // cleanup
            return 1;                                               // abort
        }
```

```
        if (trim(buf)[0] != '\0') seed = (unsigned int)strtoul(buf, NULL, 10); // user seed
    }

    if (!read_line("Which k values (dimensions) would you like to benchmark? Examples: '8', '8 12', '1-12', 'all': ", buf, sizeof(buf))) {
        dataset_free(&ds);                                          // cleanup
        return 1;                                                   // abort
    }

    int ks[MAX_K];                                                  // list of k values
    int nk = parse_k_list(buf, ks);                                 // parse k list
    if (nk <= 0) {                                                  // none parsed?
        fprintf(stderr, "No valid k values provided.\n");          // error
        dataset_free(&ds);                                          // cleanup
        return 1;                                                   // abort
    }

    if (!read_line("Compare KD-tree results to naive for correctness? (y/n): ", buf, sizeof(buf))) {
        dataset_free(&ds);                                          // cleanup
        return 1;                                                   // abort
    }
    bool do_compare = (tolower((unsigned char)trim(buf)[0]) == 'y'); // y => true

    int *all_idx = (int*)xmalloc((size_t)ds.n * sizeof(int));       // indices 0..N-1
    for (int i = 0; i < ds.n; i++) all_idx[i] = i;                  // fill indices

    if (!use_all) shuffle_ints(all_idx, ds.n, seed);                // shuffle if sampling

    int *query_idx = all_idx;                                       // queries = first Q indices in all_idx

    print_results_header();                                         // print table header

    for (int i = 0; i < nk; i++) {                                  // for each requested k
        int k = ks[i];                                              // current k
        if (k > ds.m) {                                             // cannot exceed available metrics
            printf("| %2d | (skipped: dataset has only %d metrics)\n", k, ds.m); // skip message
            continue;                                               // next k
        }
        BenchResult br = run_benchmark_for_k(&ds, query_idx, Q, k, do_compare); // run benchmark
        print_result_row(&br);                                      // print row
    }

    printf("------------------
```

For the evaluation executable, I compile the KD-tree demo file with main renamed so it can be linked as a library into the evaluation driver.

- gcc -Dmain=kd_demo_main -c Build_KD-Tree_and_NNSearch.c -o KD-Tree_no_main.o
- gcc Evaluation_Final.c KD-Tree_no_main.o Standardise_no_main.o -lm -o Run_Evaluation.o
- ./Run_Evaluation.o Data_for_Project.csv

5.3 Curse of Dimensionality

The curse of dimensionality refers to the fact that many geometric and distance-based algorithms become less effective as dimensionality increases. In the context of KD-trees, performance gains are driven by pruning: the algorithm avoids searching entire subtrees when it knows they cannot contain a closer point than the current best candidate. In low-dimensional spaces, these pruning decisions are strong because the search radius intersects relatively few partitions. In higher-dimensional spaces, partitions are less discriminating and the search area is more likely to intersect both sides of many splits, forcing the algorithm to explore far more of the tree.

This effect is directly visible in the evaluation metrics. As k rises from 1 to 12, the average number of visited nodes per query increases sharply from ~ 11.64 ($k = 1$) to ~ 613.69 ($k = 12$). Correspondingly, the estimated prune rate falls from $\sim 99.42\%$ to $\sim 69.38\%$. In basic terms, at $k = 1$ the KD-tree can discard almost the entire dataset for each query; by $k = 12$, it must examine a substantial fraction of the dataset (roughly 30% of all points, on average), which pushes KD-tree behaviour closer to a linear scan. The speedup metric mirrors this: it drops from $\sim 48\times$ at $k = 1$ to $\sim 2.71\times$ at $k = 12$. The KD-tree remains faster even at 12 dimensions for this dataset size, but the gap is much smaller because pruning power has been eroded by dimensionality.

This also aligns with the common rule-of-thumb that KD-trees degrade when N is not very large compared to 2^k. Here, $N \approx 2000$ while $2^{12} = 4096$; these are of the same order of magnitude. That is a set where the KD-tree still provides benefit, but cannot be expected to deliver the dramatic pruning seen in very low dimensions or in much larger datasets. The conclusion from the measured results is that the curse of dimensionality can be seen in practice: it shows up concretely as rising node visits, falling prune percentages, and shrinking speedups as k increases.

5.4 Other analysis

Two other points are important when interpreting these results. First, KD-tree performance must be evaluated with respect to workload. Building the tree has an upfront cost (in my results, typically a few milliseconds per chosen k), but this cost is compunded over repeated queries. If only a very small number of queries were required, a naive search might be competitive because it has no preprocessing to do. In contrast, in realistic use-cases such as peer discovery, repeated similarity lookups, or batch evaluation of many stocks, the KD-tree's build cost becomes negligible relative to total query savings.

Secondly, speedup does not strictly decrease smoothly with k. Real machines introduce effects such as caching, branch prediction, recursion overhead, and differences in query difficulty (some queries naturally cause more backtracking than others). As a result, small non-monotonic bumps in runtime can occur even when the overall trend is clear. The key indicators that the evaluation is behaving correctly are: (i) KD-tree mismatches remain zero (correctness is preserved), (ii) average visited nodes rises materially with k, and (iii) average pruned percentage falls materially with k. All three are present in the results table.

Finally, these results help position where KD-trees could be most useful. For low-dimensional subsets of metrics (e.g., $k \leq 6$ in this dataset), KD-tree pruning is strong and speedups are large. For higher-dimensional metric sets (e.g., $k \approx 12$), KD-trees still outperform brute force for $N \approx 2000$, but the advantage narrows substantially. If the project were extended to even higher dimensionality

(dozens or hundreds of features), the KD-tree would likely approach linear-scan behaviour, and alternative approaches would become more appropriate. Within the scope of this, however, the evaluation demonstrates both sides: KD-trees can be extremely effective in low dimensions, and the curse of dimensionality measurably reduces that effectiveness as k grows.

6 Applications

6.1 How could this be used on a trading floor?

- Relative value and "peer discovery" in factor space Traders routinely need a defensible peer set, but "same sector" is often too blunt. This produces a peer based on measurable characteristics (valuation, leverage, margins, growth, risk, size). Practical desk use: You would easliy be able to find the closest peer to benchmark a stock on EV/EBITDA or EV/EBIT conditioned on similar growth or margins.

  Comparable sets: Equity research / IB-style comparables can be automated. A banker can justify comps not just by "industry," but by an actual distance score over the specific multiples and profitability metrics used in a pitch or analysis. Metric subsetting ("Which metrics would you like to consider") naturally matches different trading styles.

- Hedging and proxy construction.

  On a trading floor, a common problem is finding a proxy hedge when; the most logical hedge is illiquid, a borrow is tight, a stock is halted, or you are restricted from trading a direct peer.

  This code can be used to: - Identify a nearest neighbour hedge candidate based on the factors most relevant to P&L or another metric that you are looking at. - Build a general peer basket that acts as a cleaner hedge than a single name, reducing idiosyncratic risk.

- Event-driven trading: "who should move with this?" When a stock has an earnings surprise, regulatory headline, or sector shock, desks ask: "Who is the closest analogue and did they move the same way?" "Who should have moved but didn't (potential laggard/leader trade)?"

  This code could carry out these requests.

- Operational advantages: explainability and trader trust

  On desks, a general made-up similarity score is often rejected. The code can: show the chosen metric list, and give absolute z-score differences. This is exactly the kind of explanation that helps adoption.

6.2 Next steps

- Move from "nearest neighbour" to k nearest neighbours: Right now the program returns a single closest stock. On a trading floor, the more useful output is usually a peer list (top 5 / top 10). To improve this again one could implement a distance score for each and even with a basket construction (equal-weight or distance-weighted).

  Why it matters: Single-name hedges are noisy; a peer basket hedges exposures more robustly and gives a larger set to work with. It answers questions like "top 10 comps" or "build a neutral basket."

  Implementation direction: Maintain a small fixed-size max-heap (size k) of best candidates during tree traversal instead of one.

- Add metric weighting and/or a better distance metric (finance-impact: high)

The Euclidean distance on z-scores implicitly assigns equal importance to every selected metric. In practice: A trader may care far more about beta than dividend $, while a value analyst may want EV/EBITDA to dominate.

Two common upgrades:

- Weighted Euclidean: $d(x, y) = \sqrt{\sum_{i=1}^{k} w_i \, (x_i - y_i)^2}$
- Mahalanobis distance: accounts for correlation among metrics (e.g., EBIT margin and EBITDA margin are not independent). This is closer to "risk model thinking."

- Improve data hygiene: outliers and missingness

  My standardisation fills missing values with the column mean and then z-scores. Mean-imputation shrinks the true dispersion of each feature by pulling missing observations toward the centre, which can artificially reduce distances and make firms with missing data look "average." It can also distort correlation across metrics, changing the geometry of the feature space the KD-tree relies on. For nearest-neighbour search, this may bias matches toward firms with similar missingness patterns rather than genuinely similar fundamentals. Finally, it hides the fact that data were missing at all, which can weaken interpretability and decision-making.

  Possible upgrades: include robust-scale metrics (median/MAD) before z-scoring. For missing values: note and accept missingness by having a functionality that could just ignore rather impute the mean or use more defensible imputation (more sector based)

- Performance: make the k-d tree actually pay off:

  As seen above naive scan can be only seconds behind KD-Tree for 2,000 stocks. That is expected: at only this size, constant factors dominate, the code can optimise well and in 12 dimensions the k-d tree prunes less aggressively (curse of dimensionality) as discussed above.

  To make the structure materially worthwhile in practice: answer many queries per build or a batch mode to "compute neighbours for every stock".

  Algorithmic upgrades could include: my build currently sorts at every node (qsort each recursion), which pushes me toward $O\big(n \log^2 n\big)$ Using a more basic median-selection approach per level while incoportaing some self-balancing may improve this.

7 Conclusion

This project implemented a KD-tree–based nearest-neighbour search over approximately 2,000 firms represented by 12 standardised financial metrics, and compared its performance against a brute-force linear scan baseline. By z-score standardising features, the distance metric becomes scale-consistent, enabling meaningful similarity search across financial variables. Empirical benchmarking shows that the KD-tree can deliver substantial speedups and pruning benefits in low dimensions, but performance degrades as dimensionality increases—consistent with the curse of dimensionality. Overall, the KD-tree approach is effective for fast similarity lookup when the feature set is modest and well-conditioned, while higher-dimensional settings require additional techniques (e.g., feature selection, dimensionality reduction, or approximate k-NN) to preserve pruning efficiency. Key limitations include the impact of missing-value imputation on distance geometry and the inherent weakening of KD-tree pruning as k grows, which define clear directions for future enhancement.

Appendix

A Definitions of Financial Terms

- Enterprise Multiple=EV/EBITDA Enterprise Value=Market capitalization +total debt−cash and cash equivalents. EBITDA=Earnings before interest, taxes, depreciation and amortization
- Market capitalization is the current value of a publicly traded company, based on the total dollar amount that all of its outstanding shares are worth.
- Beta is an indication of the volatility of a stock, a fund, or a stock portfolio in comparison with the market as a whole. Beta=CR/Variance of Market's ReturnCR=Covariance of asset's return with market's return
- D/E ratio = Total liabilities/ equity provided by stockholders This metric reveals the respective amounts of debt and equity a company uses to finance its operations.
- The price-to-earnings (P/E) ratio compares a company's share price with its earnings per share. Used to determine the relative value of a company's shares in side-by-side comparisons.
- Revenue growth is the increase (or decrease) in a company's sales from one period to the next.
- The gross margin is the percentage of the company's revenue that exceeds its cost of goods sold. The gross margin is calculated by subtracting the cost of goods sold (COGS) from revenue
- EBITDA margin = (earnings before interest and tax + depreciation + amortization) / total revenue
- Dividend per share is the sum of declared dividends issued by a company for every ordinary share outstanding.
- Operating margin = operating income (gross income – operating costs) / revenue
- EBIT margin = EBIT (earnings before interest and taxation) / revenue
- EV/ EBIT = Enterprise Value = Market capitalization + total debt − cash and cash equivalents. EBIT = Earnings before interest and taxes.